

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 6”

Disusun oleh:

Rifal maulana oskar

2511533024

Dosen Pengampu : Dr. Wahyudi, S.T, M.T

Asisten Praktikum : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum ini dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum Struktur Data pada pertemuan ke enam dengan topik dasar pemrograman Java, yaitu pembuatan program Doubly Linked List.

Semoga laporan ini dapat memberikan pemahaman dasar mengenai penggunaan bahasa pemrograman Java, khususnya dalam mengenal struktur dasar program, fungsi, serta pemanggilan metode.

Penulis menyadari laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Padang, 14 September 2026

Rifal maulana oskar

DAFTAR ISI

BAB I	4
1.1 Latar belakang	4
1.2 Tujuan	4
1.3 Manfaat	4
BAB II	5
2.1 Langkah kerja program NodeDLL	5
2.2 Contoh program NodeDLL.	5
2.5 Analisis hasil dan Teori :	5
2.6 Langkah kerja program InsertDLL	5
2.7 Contoh kode program InsertDLL.	6
2.8 Hasil output :	9
2.9 Analisis hasil dan teori :	9
2.10 Langkah kerja program HapusDLL.	9
2.11 contoh kode program HapusDLL.	9
2.12 Hasil output	12
2.13 Analisis hasil dan teori	13
2.14 Langkah kerja program PenelusuranDLL	13
2.15 Contoh kerja program PenulusuranDLL.	13
2.16 Hasil output	14
2.17 Analisis hasil dan teori.	15
BAB III	16
DAFTAR PUSTAKA	17

BAB I

PENDAHULUAN

1.1 Latar belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, serta memproses data secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Linked List sendiri memiliki beberapa jenis, salah satunya yaitu Doubly Linked List (DLL).

Doubly Linked List adalah jenis linked list yang memiliki dua arah koneksi, yaitu pointer next untuk menuju node berikutnya dan pointer prev untuk menuju node sebelumnya. Struktur ini mempermudah penelusuran data dari depan maupun dari belakang, serta memudahkan proses penyisipan dan penghapusan node di posisi tertentu.

Praktikum ini dilakukan untuk memahami cara kerja Doubly Linked List serta implementasinya dalam bahasa pemrograman Java.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Doubly Linked List.
2. Mengimplementasikan node Doubly Linked List pada Java.
3. Membuat program penelusuran maju dan mundur.
4. Membuat operasi penyisipan (insert) pada awal, akhir, dan posisi tertentu.
5. Membuat operasi penghapusan (delete) pada awal, akhir, dan posisi tertentu.
6. Menampilkan hasil dari setiap proses pada Doubly Linked List.

1.3 Manfaat

Pembahasan ini diharapkan dapat membantu pembaca dalam:

- Memahami sintaks dan logika DLL
- Mengimplementasikan kode syntax.

BAB II PEMBAHASAN

2.1 Langkah kerja program NodeDLL.

- Membuat kelas operator dalam package pekan6.
- Deklarasikan class utama dengan nama NodeDLL_2511533024
- Masukan kode program.

2.2 Contoh program NodeDLL.

```
package pekan6_2511533024;

public class nodedll_2511533024 {
    int data;
    nodedll_2511533024 next;
    nodedll_2511533024 prev;
    public nodedll_2511533024(int data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }
}
```

2.5 Analisis hasil dan Teori :

Ini Adalah kelas method untuk program yang akan dibuat.

2.6 Langkah kerja program InsertDLL

- Buat di kelas operator dalam package pekan6.
- Deklarasikan class utama dengan nama InsertDLL_2511533024.
- Masukan kode program perulangan
- Run program.

2.7 Contoh kode program InsertDLL.

```
package pekan6_2511533024;

public class insertdll_2511533024 {

    static nodedll_2511533024 insertBegin_3024(nodedll_2511533024 head_3024, int
data_3024) {

        nodedll_2511533024 new_node_3024 = new nodedll_2511533024(data_3024);
        new_node_3024.next = head_3024;
        if (head_3024 != null) {
            head_3024.prev = new_node_3024;
        }
        return new_node_3024;
    }

    public static nodedll_2511533024 insertEnd_3024(nodedll_2511533024 head_3024, int
newdata_3024) {

        nodedll_2511533024 newnode_3024 = new nodedll_2511533024(newdata_3024);
        if (head_3024 == null) {
            return newnode_3024;
        }
        nodedll_2511533024 curr_3024 = head_3024;
        while (curr_3024.next != null) {
            curr_3024 = curr_3024.next;
        }
        curr_3024.next = newnode_3024;
        newnode_3024.prev = curr_3024;
        return head_3024;
    }

    public static nodedll_2511533024 insertAtPosition_3024(nodedll_2511533024 head_3024,
int pos_3024, int new_data_3024) {

        if (pos_3024 <= 0) {
            System.out.println("Posisi tidak valid!");
            return head_3024;
        }
    }
}
```

```

    }
    nodedll_2511533024 new_node_3024 = new nodedll_2511533024(new_data_3024);
    if(pos_3024 == 1) {
        new_node_3024.next = head_3024;
        if(head_3024 != null) {
            head_3024.prev = new_node_3024;
        }
        return new_node_3024;
    }
    nodedll_2511533024 curr_3024 = head_3024;
    for (int i_3024 = 1; i_3024 < pos_3024 - 1 && curr_3024 != null; i_3024++) {
        curr_3024 = curr_3024.next;
    }
    if(curr_3024 == null) {
        System.out.println("Posisi tidak ada!");
        return head_3024;
    }
    new_node_3024.next = curr_3024.next;
    new_node_3024.prev = curr_3024;
    if(curr_3024.next != null) {
        curr_3024.next.prev = new_node_3024;
    }
    curr_3024.next = new_node_3024;
    return head_3024;
}

public static void printlist_3024(nodedll_2511533024 head_3024) {
    nodedll_2511533024 curr_3024 = head_3024;
    while (curr_3024 != null) {

```

```

        System.out.print(curr_3024.data);
        if (curr_3024.next != null) {
            System.out.print(" <-> ");
        }
        curr_3024 = curr_3024.next; // ini wajib biar tidak infinite loop
    }
    System.out.println();
}

public static void main(String[] args) {
    nodedll_2511533024 head_3024 = new nodedll_2511533024(2);
    head_3024.next = new nodedll_2511533024(3);
    head_3024.next.prev = head_3024;
    head_3024.next.next = new nodedll_2511533024(5);
    head_3024.next.next.prev = head_3024.next;
    System.out.print("DLL awal : ");
    printlist_3024(head_3024);
    head_3024 = insertBegin_3024(head_3024, 1);
    System.out.print("Setelah tambah node 1 di awal: ");
    printlist_3024(head_3024);
    System.out.print("Simpul 6 ditambah di akhir: ");
    head_3024 = insertEnd_3024(head_3024, 6);
    printlist_3024(head_3024);
    System.out.println("Tambah node 4 di posisi 4:");
    head_3024 = insertAtPosition_3024(head_3024, 4, 4);
    printlist_3024(head_3024);
}
}

```


2.8 Hasil output :

DLL awal : 2 <-> 3 <-> 5

Setelah tambah node 1 di awal: 1 <-> 2 <-> 3 <-> 5

Simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6

Tambah node 4 di posisi 4:

1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6

2.9 Analisis hasil dan teori :

Program insertdll_2511533024 merupakan implementasi operasi penyisipan (insert) pada struktur data **Doubly Linked List (DLL)**. DLL adalah struktur data yang setiap nodenya memiliki data, pointer next untuk menuju node berikutnya, dan pointer prev untuk menuju node sebelumnya. Pada proses insert, node baru dapat ditambahkan di awal, akhir, maupun posisi tertentu, dengan syarat pointer next dan prev harus diperbarui agar hubungan antar node tetap benar.

Berdasarkan hasil program, operasi insert berjalan sesuai teori. List awal 2 <-> 3 <-> 5 berhasil ditambah node 1 di awal sehingga menjadi 1 <-> 2 <-> 3 <-> 5. Kemudian node 6 ditambahkan di akhir sehingga list menjadi 1 <-> 2 <-> 3 <-> 5 <-> 6. Setelah itu node 4 dimasukkan pada posisi ke-4 sehingga hasil akhir menjadi 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6. Output menunjukkan bahwa pointer next dan prev sudah diatur dengan benar sehingga urutan node sesuai dan struktur list tidak rusak.

2.10 Langkah kerja program HapusDLL.

- Buat di kelas operator dalam package pekan6.
- Deklarasikan class utama dengan nama HapusDLL_2511533024.
- Masukan kode program HapusDLL_2511533024.
- Run program.

2.11 contoh kode program HapusDLL.

```
package pekan6_2511533024;  
  
public class hapusdll_2511533024 {  
    public static nodedll_2511533024 delhead_3024(nodedll_2511533024 head_3024) {  
        if (head_3024 == null) {  
            return null;  
        }  
    }  
}
```

```

        head_3024 = head_3024.next;
    if (head_3024 != null) {
        head_3024.prev = null;
    }
    return head_3024;
}

public static nodedll_2511533024 dellast_3024(nodedll_2511533024 head_3024) {
    if (head_3024 == null) {
        return null;
    }
    if (head_3024.next == null) {
        return null;
    }
    nodedll_2511533024 curr_3024 = head_3024;
    while (curr_3024.next != null) {
        curr_3024 = curr_3024.next;
    }
    if (curr_3024.prev != null) {
        curr_3024.prev.next = null;
    }
    return head_3024;
}

public static nodedll_2511533024 delpos_3024(nodedll_2511533024 head_3024, int
pos_3024) {
    if (head_3024 == null || pos_3024 <= 0) {
        return head_3024;
    }
    nodedll_2511533024 curr_3024 = head_3024;
    // Jika yang dihapus adalah posisi 1 (head)
    if (pos_3024 == 1) {

```

```

        return delhead_3024(head_3024);
    }
    for (int i_3024 = 1; curr_3024 != null && i_3024 < pos_3024; i_3024++) {
        curr_3024 = curr_3024.next;
    }
    if (curr_3024 == null) {
        return head_3024;
    }
    if (curr_3024.prev != null) {
        curr_3024.prev.next = curr_3024.next;
    }
    if (curr_3024.next != null) {
        curr_3024.next.prev = curr_3024.prev;
    }
    return head_3024;
}

public static void printList_3024(nodedll_2511533024 head_3024) {
    nodedll_2511533024 curr_3024 = head_3024;
    while (curr_3024 != null) {
        System.out.print(curr_3024.data + " ");
        curr_3024 = curr_3024.next;
    }
    System.out.println();
}

public static void main(String[] args) {
    nodedll_2511533024 head_3024 = new nodedll_2511533024(1);
    head_3024.next = new nodedll_2511533024(2);
    head_3024.next.prev = head_3024;
    head_3024.next.next = new nodedll_2511533024(3);

```

```

    head_3024.next.next.prev = head_3024.next;
    head_3024.next.next.next = new nodedll_2511533024(4);
    head_3024.next.next.next.prev = head_3024.next.next;
    head_3024.next.next.next.next = new nodedll_2511533024(5);
    head_3024.next.next.next.next.prev = head_3024.next.next.next;
    System.out.print("DLL awal: ");
    printList_3024(head_3024);
    System.out.print("setelah head dihapus: ");
    head_3024 = delhead_3024(head_3024);
    printList_3024(head_3024);
    System.out.print("Setelah node terakhir dihapus: ");
    head_3024 = dellast_3024(head_3024);
    printList_3024(head_3024);
    System.out.print("Setelah menghapus node ke-2: ");
    head_3024 = delpos_3024(head_3024, 2);
    printList_3024(head_3024);
}
}

```

2.12 Hasil output

DLL awal: 1 2 3 4 5

setelah head dihapus: 2 3 4 5

Setelah node terakhir dihapus: 2 3 4

Setelah menghapus node ke-2: 2 4

2.13 Analisis hasil dan teori

Program hapusdll_2511533024 merupakan implementasi operasi penghapusan (delete) pada struktur data **Doubly Linked List (DLL)**. Doubly Linked List adalah struktur data yang setiap nodenya memiliki data, pointer next yang menunjuk node berikutnya, dan pointer prev yang menunjuk node sebelumnya. Pada operasi delete, node dapat dihapus dari beberapa posisi seperti node pertama (head), node terakhir (tail), maupun node pada posisi tertentu. Proses penghapusan pada DLL harus dilakukan dengan memperbarui pointer next dan prev pada node di sekitarnya agar rantai list tetap tersambung dan tidak terjadi error.

Berdasarkan hasil program, operasi delete berjalan dengan baik sesuai teori. DLL awal berisi 1 2 3 4 5. Ketika dilakukan penghapusan head menggunakan method delhead_3024, node pertama berhasil dihapus sehingga list menjadi 2 3 4 5 karena head dipindahkan ke node berikutnya dan pointer prev dihapus (null). Setelah itu penghapusan node terakhir menggunakan method dellast_3024 berhasil menghapus tail sehingga list menjadi 2 3 4 dengan cara memutus hubungan node terakhir dari node sebelumnya. Selanjutnya penghapusan node pada posisi ke-2 menggunakan method delpos_3024 berhasil menghapus node 3, sehingga hasil akhir list menjadi 2 4. Output program membuktikan bahwa setiap proses delete sudah mengatur ulang pointer dengan benar sehingga list tetap terhubung dan tidak terjadi kerusakan struktur data.

2.14 Langkah kerja program PenelusuranDLL

- Buat di kelas operator dalam package pekan6.
- Deklarasikan class utama dengan nama PenulusuranDLL_2511533024.
- Masukan kode program PenuusuranDLL_2511533024
- Run program.

2.15 Contoh kerja program PenulusuranDLL.

```
package pekan6_2511533024;

public class penulusurandll_2511533024 {

    static void forwardTraversal(nodedll_2511533024 head) {

        nodedll_2511533024 curr = head;

        while (curr != null) {

            System.out.print(curr.data + " <-> ");

            curr = curr.next;

        }

        System.out.println();

    }

}
```

```

static void backwardTraversal(nodedll_2511533024 tail) {
    nodedll_2511533024 curr = tail;
    while (curr != null) {
        System.out.print(curr.data + " <-> ");
        curr = curr.prev;
    }
    System.out.println();
}

public static void main (String [] args) {
    nodedll_2511533024 head = new nodedll_2511533024(1);
    nodedll_2511533024 second = new nodedll_2511533024(2);
    nodedll_2511533024 third = new nodedll_2511533024(3);
    head.next = second;
    second.prev = head;
    second.next = third;
    third.prev = second;
    System.out.println("penelusuran maju : ");
    forwardTraversal(head);
    System.out.println("penelusuran mundur : ");
    backwardTraversal(third);
}
}

```

2.16 Hasil output

penelusuran maju :

1 <-> 2 <-> 3 <->

penelusuran mundur :

3 <-> 2 <-> 1 <->

2.17 Analisis hasil dan teori.

Program penulusurandll_2511533024 merupakan implementasi operasi penelusuran (traversal) pada struktur data **Doubly Linked List (DLL)**. Doubly Linked List adalah struktur data linear yang setiap nodenya memiliki data, pointer next untuk mengarah ke node berikutnya, serta pointer prev untuk mengarah ke node sebelumnya. Dengan adanya dua pointer tersebut, DLL memungkinkan proses traversal dilakukan dalam dua arah, yaitu traversal maju dari head ke tail dan traversal mundur dari tail ke head. Traversal digunakan untuk membaca atau menampilkan seluruh isi node yang ada dalam linked list secara berurutan.

Berdasarkan hasil program, penelusuran maju dan mundur berjalan sesuai teori. Pada traversal maju menggunakan method forwardTraversal, program memulai dari node head lalu berpindah dari satu node ke node berikutnya menggunakan pointer next sampai mencapai nilai null. Output yang dihasilkan adalah 1 <-> 2 <-> 3 <-> yang menunjukkan node berhasil ditampilkan dari awal hingga akhir. Sedangkan pada traversal mundur menggunakan method backwardTraversal, program memulai dari node terakhir (tail) lalu bergerak ke node sebelumnya menggunakan pointer prev hingga mencapai null. Output yang dihasilkan adalah 3 <-> 2 <-> 1 <-> yang membuktikan traversal mundur berhasil dilakukan. Dengan demikian, program ini menunjukkan bahwa struktur DLL telah terhubung dengan benar karena dapat ditelusuri dari dua arah tanpa error.

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa **Doubly Linked List (DLL)** merupakan struktur data linear yang setiap node-nya memiliki dua pointer yaitu next untuk menunjuk node berikutnya dan prev untuk menunjuk node sebelumnya, sehingga penelusuran data dapat dilakukan secara maju maupun mundur. Pada praktikum ini, program berhasil mengimplementasikan operasi traversal (forward dan backward), operasi penyisipan (insert) di awal, akhir, dan posisi tertentu, serta operasi penghapusan (delete) pada head, tail, dan posisi tertentu. Hasil output menunjukkan bahwa setiap operasi berjalan dengan baik karena pointer next dan prev selalu diperbarui dengan benar sehingga struktur list tetap terhubung dan tidak mengalami error. Dengan demikian, praktikum ini membuktikan bahwa DLL sangat efektif digunakan untuk pengolahan data yang membutuhkan akses dua arah serta mempermudah proses penambahan dan penghapusan node.

DAFTAR PUSTAKA

Modul Praktikum Struktur Data – Materi Doubly Linked List.

Abdul Kadir. (2013). *Algoritma dan Struktur Data Menggunakan Java*. Andi Publisher.

Oracle Documentation. *Java Programming Language*.